

1. Introducción

1.1 Propósito del documento

Este documento describe el conjunto de casos de prueba unitaria diseñados para validar el correcto funcionamiento de los módulos críticos del sistema SmartFlow, con el fin de garantizar la calidad, seguridad y estabilidad del código antes de su despliegue. Las pruebas cubren tanto la lógica de negocio del backend (servicios, estrategias de autenticación, guards) como las validaciones de formularios en el frontend mediante schemas Zod.

1.2 Alcance

Los casos de prueba documentados abarcan los siguientes módulos del sistema:

- Módulo de Autenticación: login, cambio de contraseña, validación de tokens JWT y guards de autorización.
- Módulo de Clientes: creación, búsqueda, actualización y desactivación de clientes.
- Módulo de Vista General (Dashboard): cálculo de métricas, ingresos, variaciones porcentuales y alertas de stock bajo.

Quedan fuera del alcance las pruebas de integración end-to-end, pruebas de carga/rendimiento y pruebas de interfaz de usuario (UI), las cuales serán documentadas en informes independientes.

1.3 Objetivos

- Verificar que cada método de los servicios analizados se comporte según lo esperado bajo condiciones normales y de error.
- Asegurar el manejo correcto de excepciones (UnauthorizedException, BadRequestException, NotFoundException) ante datos inválidos o inconsistentes.
- Validar las reglas de negocio relacionadas con duplicidad de datos, estados de entidades y cálculos derivados.
- Confirmar que los schemas de validación del frontend (Zod) rechacen y acepten datos correctamente según las reglas definidas.

1.4 Entorno y herramientas

- Framework de backend: NestJS
- Framework de pruebas: Jest
- Estrategia de pruebas: pruebas unitarias con mocks de repositorios (TypeORM) y dependencias externas (bcrypt, JWT)
- Validación frontend: Zod
- Base de datos: PostgreSQL (mockeada durante las pruebas)

1.5 Convenciones y nomenclatura

Cada caso de prueba se identifica con un código compuesto por el prefijo del módulo (AUTH, CLI, OVW) seguido de un número secuencial (ej. CP-AUTH-01). Cada caso incluye:

- ID: identificador único del caso.
- Módulo: componente del sistema bajo prueba.
- Método: función o método específico evaluado.
- Prioridad: nivel de importancia (Alta, Media, Baja) según el impacto en el sistema.
- Precondiciones: estado previo requerido para ejecutar la prueba.
- Datos de entrada: valores utilizados para la ejecución.
- Pasos: secuencia de acciones/mocks necesarios.
- Resultado esperado: comportamiento o salida que debe producirse.

1.6 Niveles de prioridad

Prioridad	Criterio
Alta	Funcionalidad crítica para la seguridad, integridad de datos o reglas de negocio esenciales
Media	Funcionalidad importante pero con menor impacto si falla temporalmente
Baja	Validaciones secundarias o casos de borde poco frecuentes

CP-AUTH-01 — Login exitoso con credenciales válidas *(continuación)*

Resultado esperado:

typescript

```
{
  access_token: 'mock_token',
  admin: {
    id: '550e8400-e29b-41d4-a716-446655440000',
    nombre: 'Admin Test',
    email: 'admin@test.com',
  }
}
```

Resultado obtenido: El sistema retorna el token JWT y los datos básicos del administrador sin incluir la contraseña.

CP-AUTH-02 — Login fallido con contraseña incorrecta

Campo	Detalle
ID	CP-AUTH-02
Módulo	AuthService
Método	login(dto: LoginDto)
Prioridad	Alta

Precondiciones:

- Existe un administrador activo con email `admin@test.com`

Datos de entrada:

typescript:

```
const dto: LoginDto = {  
  email: 'admin@test.com',  
  password: 'contraseña_incorrecta',  
};
```

Pasos:

1. Mockear `AdministradorService.findByEmail` para retornar el admin
2. Mockear `bcrypt.compare` para retornar `false`
3. Llamar `authService.login(dto)`

Resultado esperado:

- Se lanza `UnauthorizedException` con mensaje "`Credenciales inválidas`"
- No se genera ningún token JWT

CP-AUTH-03 — Login fallido con email no registrado

Campo	Detalle
ID	CP-AUTH-03
Módulo	AuthService

Método `login(dto:
LoginDto)`

Prioridad Alta

Precondiciones:

- No existe ningún administrador con el email proporcionado

Datos de entrada:

typescript

```
const dto: LoginDto = {  
  email: 'noexiste@test.com',  
  password: 'password123',  
};
```

Pasos:

1. Mockear `AdministradorService.findByEmail` para retornar `null`
2. Llamar `authService.login(dto)`

Resultado esperado:

- Se lanza `UnauthorizedException` con mensaje "`Credenciales inválidas`"

CP-AUTH-04 — Login fallido con cuenta desactivada

Campo	Detalle
ID	CP-AUTH-04
Módulo	AuthService
Método	<code>login(dto: LoginDto)</code>
Prioridad	Alta

Precondiciones:

- Existe un administrador con email `admin@test.com` pero con `activo: false`

Datos de entrada:

typescript

```
const dto: LoginDto = {  
  email: 'admin@test.com',  
  password: 'password123',  
};
```

Pasos:

1. Mockear `AdministradorService.findByEmail` para retornar admin con `activo: false`
2. Llamar `authService.login(dto)`

Resultado esperado:

- Se lanza `UnauthorizedException` con mensaje "Cuenta desactivada"

2.2 AdministradorService — Cambio de contraseña

CP-AUTH-05 — Cambio de contraseña exitoso

Campo	Detalle
ID	CP-AUTH-05
Módulo	AdministradorService
Método	<code>changePassword(id, dto)</code>
Prioridad	Alta

Precondiciones:

- Existe un administrador con el ID proporcionado
- La contraseña actual es correcta

Datos de entrada:

typescript

```
const dto: ChangePasswordDto = {  
  passwordActual: 'password123',  
  passwordNuevo: 'nuevaPassword456',  
  passwordConfirm: 'nuevaPassword456',  
};
```

Pasos:

1. Mockear `findOne` para retornar el admin con hash de contraseña

2. Mockear `bcrypt.compare` para retornar `true`
3. Mockear `bcrypt.hash` para retornar nuevo hash
4. Llamar `administradorService.changePassword(id, dto)`

Resultado esperado:

- La contraseña se actualiza correctamente
- `administradorRepository.save` es llamado con el nuevo hash
- No se lanza ninguna excepción

CP-AUTH-06 — Cambio de contraseña fallido — contraseña actual incorrecta

Campo	Detalle
ID	CP-AUTH-06
Módulo	AdministradorService
Método	<code>changePassword(id, dto)</code>
Prioridad	Alta

Precondiciones:

- Existe un administrador con el ID proporcionado

Datos de entrada:

typescript

```
const dto: ChangePasswordDto = {  
  passwordActual: 'incorrecta',  
  passwordNuevo: 'nuevaPassword456',  
  passwordConfirm: 'nuevaPassword456',  
};
```

Pasos:

1. Mockear `findOne` para retornar el admin
2. Mockear `bcrypt.compare` para retornar `false`
3. Llamar `administradorService.changePassword(id, dto)`

Resultado esperado:

- Se lanza `BadRequestException` con mensaje "La contraseña actual es incorrecta"

CP-AUTH-07 — Cambio de contraseña fallido — confirmación no coincide

Campo	Detalle
ID	CP-AUTH-07
Módulo	AdministradorService
Método	<code>changePassword(id, dto)</code>
Prioridad	Alta

Precondiciones:

- Existe un administrador con el ID proporcionado

Datos de entrada:

typescript

```
const dto: ChangePasswordDto = {
  passwordActual: 'password123',
  passwordNuevo: 'nuevaPassword456',
  passwordConfirm: 'diferente789',
};
```

Pasos:

1. Llamar `administradorService.changePassword(id, dto)`

Resultado esperado:

- Se lanza `BadRequestException` con mensaje "Las contraseñas no coinciden"
- `bcrypt.compare` nunca es llamado

2.3 JwtStrategy

CP-AUTH-08 — Validación exitosa de token JWT

Campo	Detalle
ID	CP-AUTH-08
Módulo	JwtStrategy
Método	<code>validate(payload)</code>

Prioridad Alta

Precondiciones:

- El payload JWT corresponde a un administrador activo existente

Datos de entrada:

typescript

```
const payload = {
  sub: '550e8400-e29b-41d4-a716-446655440000',
  email: 'admin@test.com',
  nombre: 'Admin Test',
};
```

Pasos:

1. Mockear `AdministradorService.findOne` para retornar admin con `activo: true`
2. Llamar `jwtStrategy.validate(payload)`

Resultado esperado:

typescript

```
{
  id: '550e8400-e29b-41d4-a716-446655440000',
  email: 'admin@test.com',
  nombre: 'Admin Test',
}
```

CP-AUTH-09 — Validación fallida — administrador inactivo

Campo	Detalle
ID	CP-AUTH-09
Módulo	JwtStrategy
Método	<code>validate(payload)</code>
Prioridad	Alta

Precondiciones:

- El payload JWT corresponde a un administrador con `activo: false`

Datos de entrada:

typescript

```
const payload = {  
  sub: '550e8400-e29b-41d4-a716-446655440000',  
  email: 'admin@test.com',  
  nombre: 'Admin Test',  
};
```

Pasos:

1. Mockear `AdministradorService.findOne` para retornar admin con `activo: false`
2. Llamar `jwtStrategy.validate(payload)`

Resultado esperado:

- Se lanza `UnauthorizedException` con mensaje "Token inválido"

2.4 JwtAuthGuard

CP-AUTH-10 — Ruta pública no requiere token

Campo	Detalle
ID	CP-AUTH-10
Módulo	JwtAuthGuard
Método	<code>canActivate(context)</code>
Prioridad	Alta

Precondiciones:

- El endpoint está decorado con `@Public()`

Pasos:

1. Mockear `Reflector.getAllAndOverride` para retornar `true`
2. Llamar `jwtAuthGuard.canActivate(mockContext)`

Resultado esperado:

- Retorna `true` sin verificar el token
- `super.canActivate` nunca es llamado

CP-AUTH-11 — Ruta protegida con API Key válida

Campo	Detalle
ID	CP-AUTH-11
Módulo	JwtAuthGuard
Método	<code>canActivate(context)</code>
Prioridad	Alta

Precondiciones:

- `process.env.AGENT_API_KEY` está definida como `"test_api_key"`
- La request incluye el header `x-api-key: test_api_key`

Pasos:

1. Mockear el contexto HTTP con el header `x-api-key`
2. Llamar `jwtAuthGuard.canActivate(mockContext)`

Resultado esperado:

- Retorna `true` sin verificar JWT
- `super.canActivate` nunca es llamado

CP-AUTH-12 — Ruta protegida sin token ni API Key

Campo	Detalle
ID	CP-AUTH-12
Módulo	JwtAuthGuard
Método	<code>canActivate(context)</code>
Prioridad	Alta

Precondiciones:

- La request no incluye `Authorization` ni `x-api-key`

Pasos:

1. Mockear el contexto HTTP sin headers de autenticación
2. Llamar `jwtAuthGuard.canActivate(mockContext)`

Resultado esperado:

- Se lanza `UnauthorizedException`
- El acceso al recurso es denegado

2.5 Validación de schemas — Frontend

CP-AUTH-13 — Schema de login válido

Campo	Detalle
ID	CP-AUTH-13
Módulo	Frontend — Zod Schema
Schema	<code>loginSchema</code>
Prioridad	Media

Precondiciones:

- Schema definido con email válido y contraseña mínima de 8 caracteres

Datos de entrada:

typescript

```
const input = {  
  email: 'admin@test.com',  
  password: 'password123',  
};
```

Resultado esperado:

- `schema.safeParse(input).success === true`

CP-AUTH-14 — Schema de login inválido — email malformado

Campo	Detalle
ID	CP-AUTH-14
Módulo	Frontend — Zod Schema
Schema	<code>loginSchema</code>

Prioridad Media

Datos de entrada:

typescript

```
const input = {  
  email: 'no-es-un-email',  
  password: 'password123',  
};
```

Resultado esperado:

- `schema.safeParse(input).success === false`
- Error en campo `email`: "Email inválido"

CP-AUTH-15 — Schema de cambio de contraseña — confirmación no coincide

Campo	Detalle
ID	CP-AUTH-15
Módulo	Frontend — Zod Schema
Schema	<code>changePasswordSchema</code>
Prioridad	Media

Datos de entrada:

typescript

```
const input = {  
  passwordActual: 'password123',  
  passwordNuevo: 'nueva456',  
  passwordConfirm: 'diferente789',  
};
```

Resultado esperado:

- `schema.safeParse(input).success === false`
- Error en campo `passwordConfirm`: "Las contraseñas no coinciden"

3. Módulo de Clientes

3.1 ClienteService

CP-CLI-01 — Crear cliente exitosamente

Campo	Detalle
ID	CP-CLI-01
Módulo	ClienteService
Método	<code>create(dto)</code>
Prioridad	Alta

Precondiciones:

- No existe ningún cliente con el teléfono ni email proporcionados

Datos de entrada:

typescript

```
const dto: CreateClienteDto = {  
  nombre: 'Juan Pérez',  
  telefono: '3001234567',  
  email: 'juan@test.com',  
  estado: EstadoCliente.ACTIVO,  
};
```

Pasos:

1. Mockear `clienteRepository.findOneBy({ telefono })` para retornar `null`
2. Mockear `clienteRepository.findOneBy({ email })` para retornar `null`
3. Mockear `clienteRepository.create` para retornar el cliente
4. Mockear `clienteRepository.save` para retornar el cliente guardado
5. Llamar `clienteService.create(dto)`

Resultado esperado:

- Retorna el cliente creado con todos sus campos
- `clienteRepository.save` es llamado exactamente una vez

CP-CLI-02 — Crear cliente fallido — teléfono duplicado

Campo	Detalle
ID	CP-CLI-02
Módulo	ClienteService

Método `create(dto
)`

Prioridad Alta

Precondiciones:

- Existe un cliente con teléfono `3001234567`

Datos de entrada:

typescript

```
const dto: CreateClienteDto = {  
  nombre: 'Pedro López',  
  telefono: '3001234567',  
  email: 'pedro@test.com',  
};
```

Pasos:

1. Mockear `clienteRepository.findOneBy({ telefono })` para retornar un cliente existente
2. Llamar `clienteService.create(dto)`

Resultado esperado:

- Se lanza `BadRequestException` con mensaje "El teléfono `3001234567` ya está registrado"
- `clienteRepository.save` nunca es llamado

CP-CLI-03 — Crear cliente fallido — email duplicado

Campo	Detalle
ID	CP-CLI-03
Módulo	ClienteService
Método	<code>create(dto)</code>
Prioridad	Alta

Precondiciones:

- Existe un cliente con email `juan@test.com`

Datos de entrada:

typescript

```
const dto: CreateClienteDto = {  
  nombre: 'Ana García',  
  telefono: '3009876543',  
  email: 'juan@test.com',  
};
```

Pasos:

1. Mockear `clienteRepository.findOneBy({ telefono })` para retornar `null`
2. Mockear `clienteRepository.findOneBy({ email })` para retornar cliente existente
3. Llamar `clienteService.create(dto)`

Resultado esperado:

- Se lanza `BadRequestException` con mensaje "El email `juan@test.com` ya está registrado"

CP-CLI-04 — Buscar o crear cliente — cliente existente por teléfono

Campo	Detalle
ID	CP-CLI-04
Módulo	ClienteService
Método	<code>findOrCreate(nombre, telefono, email, direccionPrincipal)</code>
Prioridad	Alta

Precondiciones:

- Existe un cliente con teléfono `3001234567`

Datos de entrada:

typescript

```
nombre = 'Juan Pérez';  
telefono = '3001234567';  
email = 'juan@test.com';  
direccionPrincipal = 'Calle 5 #10-20';
```

Pasos:

1. Mockear `clienteRepository.findOneBy({ telefono })` para retornar cliente existente sin `direccionPrincipal`
2. Llamar `clienteService.findOrCreate(nombre, telefono, email, direccionPrincipal)`

Resultado esperado:

typescript

```
{ cliente: clienteExistente, esNuevo: false }
```

- La `direccionPrincipal` es actualizada en el cliente existente
- `clienteRepository.save` es llamado para guardar la dirección

CP-CLI-05 — Buscar o crear cliente — cliente nuevo

Campo	Detalle
ID	CP-CLI-05
Módulo	ClienteService
Método	<code>findOrCreate(nombre, telefono, email, direccionPrincipal)</code>
Prioridad	Alta

Precondiciones:

- No existe ningún cliente con el teléfono ni email proporcionados

Datos de entrada:

typescript

```
nombre = 'María Torres';  
telefono = '3119876543';  
email = undefined;  
direccionPrincipal = undefined;
```

Pasos:

1. Mockear `clienteRepository.findOneBy({ telefono })` para retornar `null`
2. Mockear `clienteRepository.create` y `save` para retornar el nuevo cliente
3. Llamar `clienteService.findOrCreate(nombre, telefono)`

Resultado esperado:

typescript


```
{ cliente: nuevoCliente, esNuevo: true }
```

- El cliente es creado con `estado: EstadoCliente.ACTIVO`

CP-CLI-06 — Obtener cliente por ID existente

Campo	Detalle
ID	CP-CLI-06
Módulo	ClienteService
Método	<code>findOne(id)</code>
Prioridad	Media

Precondiciones:

- Existe un cliente con el ID proporcionado

Datos de entrada:

typescript

```
const id = '550e8400-e29b-41d4-a716-446655440000';
```

Pasos:

1. Mockear `clienteRepository.findOne` para retornar el cliente con sus pedidos
2. Llamar `clienteService.findOne(id)`

Resultado esperado:

- Retorna el cliente con la relación `pedidos` cargada
- No se lanza ninguna excepción

CP-CLI-07 — Obtener cliente por ID inexistente

Campo	Detalle
ID	CP-CLI-07
Módulo	ClienteService
Método	<code>findOne(id)</code>

Prioridad Media

Precondiciones:

- No existe ningún cliente con el ID proporcionado

Datos de entrada:

typescript

```
const id = '550e8400-e29b-41d4-a716-446655440000';
```

Pasos:

1. Mockear `clienteRepository.findOne` para retornar `null`
2. Llamar `clienteService.findOne(id)`

Resultado esperado:

- Se lanza `NotFoundException` con mensaje "`Cliente 550e8400-e29b-41d4-a716-446655440000 no encontrado`"

CP-CLI-7 — Actualizar cliente — teléfono en uso por otro cliente

Campo	Detalle
ID	CP-CLI-10
Módulo	ClienteService
Método	<code>update(id, dto)</code>
Prioridad	Alta

Precondiciones:

- Existe cliente A con ID `aaa` y cliente B con teléfono `3001234567`

Datos de entrada:

typescript

```
const id = 'aaa';
const dto: UpdateClienteDto = {
  telefono: '3001234567',
};
```

Pasos:

1. Mockear `clienteRepository.findOneBy({ telefono })` para retornar cliente B con ID diferente
2. Llamar `clienteService.update(id, dto)`

Resultado esperado:

- Se lanza `BadRequestException` con mensaje "El teléfono ya está en uso"
- `clienteRepository.save` nunca es llamado

3.2 Validación de schemas — Frontend

CP-CLI-8 — Schema de creación de cliente válido

Campo	Detalle
ID	CP-CLI-11
Módulo	Frontend — Zod Schema
Schema	<code>createClienteSchema</code>
Prioridad	Media

Datos de entrada:

typescript

```
const input = {  
  nombre: 'Juan Pérez',  
  telefono: '3001234567',  
  email: 'juan@test.com',  
  direccionPrincipal: 'Calle 5 #10-20, Popayán',  
  estado: 'ACTIVO',  
};
```

Resultado esperado:

- `schema.safeParse(input).success === true`

CP-CLI-9 — Schema de creación — nombre muy corto

Campo	Detalle
-------	---------

ID	CP-CLI-12
Módulo	Frontend — Zod Schema
Schema	<code>createClienteSchem a</code>
Prioridad	Media

Datos de entrada:

typescript

```
const input = {
  nombre: 'J',
  telefono: '3001234567',
};
```

Resultado esperado:

- `schema.safeParse(input).success === false`
- Error en campo `nombre`: "El nombre debe tener al menos 2 caracteres"

CP-CLI-10 — Schema de creación — teléfono muy corto

Campo	Detalle
ID	CP-CLI-13
Módulo	Frontend — Zod Schema
Schema	<code>createClienteSchem a</code>
Prioridad	Media

Datos de entrada:

typescript

```
const input = {
  nombre: 'Juan Pérez',
  telefono: '300',
};
```

Resultado esperado:

- `schema.safeParse(input).success === false`
- Error en campo `telefono`: "El teléfono debe tener al menos 7 dígitos"

CP-CLI-11 — Schema de creación — email inválido

Campo	Detalle
ID	CP-CLI-14
Módulo	Frontend — Zod Schema
Schema	<code>createClienteSchema</code>
Prioridad	Media

Datos de entrada:

typescript

```
const input = {  
  nombre: 'Juan Pérez',  
  telefono: '3001234567',  
  email: 'no-es-email',  
};
```

Resultado esperado:

- `schema.safeParse(input).success === false`
- Error en campo `email`: "Email inválido"

CP-CLI-12 — Schema de filtros — estado válido

Campo	Detalle
ID	CP-CLI-15
Módulo	Frontend — Zod Schema
Schema	<code>clienteFiltersSchema</code>
Prioridad	Baja

Datos de entrada:

typescript

```
const input = {  
  page: 1,  
  limit: 10,  
  estado: 'ACTIVO',  
  sortBy: 'creadoEn',  
  sortOrder: 'desc',  
};
```

Resultado esperado:

- `schema.safeParse(input).success === true`

4. Módulo de Vista General (Dashboard Overview)

4.1 DashboardService

CP-OVW-01 — Obtener overview con datos completos

Campo	Detalle
ID	CP-OVW-01
Módulo	DashboardService
Método	<code>getOverview()</code>
Prioridad	Alta

Precondiciones:

- Existen pedidos, clientes y productos en la base de datos
- Hay pedidos en estado **ENTREGADO** con fecha de hoy
- Hay productos con stock menor o igual a 5

Pasos:

1. Mockear `pedidoRepository.createQueryBuilder` para retornar conteos simulados
2. Mockear `clienteRepository.count` para retornar 8
3. Mockear `productoRepository.createQueryBuilder` para retornar 2 productos con stock bajo
4. Llamar `dashboardService.getOverview()`

Resultado esperado:

typescript

```
{
  pedidos: {
    hoy: 3,
    pendientes: 5,
    completadosTotal: 2,
  },
  ingresos: {
    hoy: 124000,
    esteMes: 580000,
    variacion: expect.any(Number),
    tendencia: expect.stringMatching(/^(up|down)$/),
  },
  clientes: {
    total: 8,
    nuevosEsteMes: expect.any(Number),
  },
  productosStockBajo: expect.arrayContaining([
    expect.objectContaining({ stock: expect.any(Number) })
  ]),
  pedidosRecientes: expect.any(Array),
}
```

CP-OVW-02 — Cálculo de variación positiva vs mes anterior

Campo	Detalle
ID	CP-OVW-02
Módulo	DashboardService
Método	<code>getOverview()</code>
Prioridad	Alta

Precondiciones:

- Ingresos del mes actual: `$600.000`
- Ingresos del mes anterior: `$500.000`

Pasos:

1. Mockear las queries de ingresos con los valores definidos
2. Llamar `dashboardService.getOverview()`

Resultado esperado:

typescript

```
ingresos: {  
  esteMes: 600000,  
  variacion: 20,    // ((600000 - 500000) / 500000) * 100  
  tendencia: 'up',  
}
```

CP-OVW-03 — Cálculo de variación negativa vs mes anterior

Campo	Detalle
ID	CP-OVW-03
Módulo	DashboardService
Método	<code>getOverview()</code>
Prioridad	Alta

Precondiciones:

- Ingresos del mes actual: `$400.000`
- Ingresos del mes anterior: `$500.000`

Pasos:

1. Mockear las queries de ingresos con los valores definidos
2. Llamar `dashboardService.getOverview()`

Resultado esperado:

typescript

```
ingresos: {  
  esteMes: 400000,  
  variacion: -20,    // ((400000 - 500000) / 500000) * 100  
  tendencia: 'down',  
}
```

CP-OVW-04 — Variación con mes anterior en cero

Campo	Detalle
ID	CP-OVW-04

Módulo DashboardService

Método `getOverview()`

Prioridad Media

Precondiciones:

- Ingresos del mes anterior: `$0` (sin ventas)
- Ingresos del mes actual: `$300.000`

Pasos:

1. Mockear query de mes anterior para retornar `{ total: null }`
2. Llamar `dashboardService.getOverview()`

Resultado esperado:

typescript

```
ingresos: {  
  variacion: 100, // cuando mes anterior es 0, variación es 100%  
  tendencia: 'up',  
}
```

CP-OVW-05 — Overview sin pedidos del día

Campo	Detalle
ID	CP-OVW-05
Módulo	DashboardService
Método	<code>getOverview()</code>
Prioridad	Media

Precondiciones:

- No hay pedidos creados hoy
- No hay ventas completadas hoy

Pasos:

1. Mockear query de pedidos hoy para retornar `0`
2. Mockear query de ingresos hoy para retornar `{ total: null }`

3. Llamar `dashboardService.getOverview()`

Resultado esperado:

typescript

pedidos: { hoy: 0, ... },
ingresos: { hoy: 0, ... },

- No se lanza ninguna excepción
- Los valores nulos se convierten a `0`

CP-OVW-06 — Productos con stock crítico correctamente identificados

Campo	Detalle
ID	CP-OVW-06
Módulo	DashboardService
Método	<code>getOverview()</code>
Prioridad	Alta

Precondiciones:

- Existen productos con stocks: 1, 2, 3, 4, 5, 10, 20

Pasos:

1. Mockear `productoRepository.createQueryBuilder` para retornar solo productos con stock ≤ 5 , ordenados ASC
2. Llamar `dashboardService.getOverview()`

Resultado esperado:

- `productosStockBajo` contiene exactamente los productos con stock 1, 2, 3, 4, 5
- Están ordenados de menor a mayor stock
- El producto con stock 10 y 20 no aparecen
- Máximo 5 productos retornados

CP-OVW-07 — Pedidos recientes ordenados por fecha descendente

Campo	Detalle
-------	---------

ID	CP-OVW-07
Módulo	DashboardService
Método	<code>getOverview()</code>
Prioridad	Media

Precondiciones:

- Existen más de 5 pedidos en la base de datos

Pasos:

1. Mockear `pedidoRepository.createQueryBuilder` para retornar 5 pedidos ordenados por `creadoEn DESC`
2. Llamar `dashboardService.getOverview()`

Resultado esperado:

- `pedidosRecientes` contiene exactamente 5 pedidos
- El primer pedido tiene la fecha más reciente
- Cada pedido incluye las relaciones `cliente` e `items`

4.2 Frontend — Schema de Overview

CP-OVW-08 — Schema de overview válido

Campo	Detalle
ID	CP-OVW-08
Módulo	Frontend — Zod Schema
Schema	<code>overviewSchema</code>
Prioridad	Media

Datos de entrada:

typescript

```
const input = {
  pedidos: { hoy: 3, pendientes: 5, completadosTotal: 2 },
  ingresos: { hoy: 124000, esteMes: 580000, variacion: 20, tendencia: 'up' },
  clientes: { total: 8, nuevosEsteMes: 3 },
  productosStockBajo: [{ id: 'uuid', nombre: 'Agua', stock: 2 }],
```

```
pedidosRecientes: [],  
};
```

Resultado esperado:

- `schema.safeParse(input).success === true`

CP-OVW-09 — Schema rechaza tendencia inválida

Campo	Detalle
ID	CP-OVW-09
Módulo	Frontend — Zod Schema
Schema	<code>overviewSchema</code>
Prioridad	Baja

Datos de entrada:

```
typescript  
ingresos: {  
  hoy: 0,  
  esteMes: 0,  
  variacion: 0,  
  te  
ndencia: 'lateral', // valor inválido  
}
```

Resultado esperado:

- `schema.safeParse(input).success === false`
- Error en campo `tendencia`: valor no permitido por el enum `[ 'up', 'down' ]`

5. Resumen de casos de prueba

ID	Módulo	Descripción	Prioridad
CP-AUTH-01	AuthService	Login exitoso	Alta
CP-AUTH-02	AuthService	Login — contraseña incorrecta	Alta
CP-AUTH-03	AuthService	Login — email no registrado	Alta
CP-AUTH-04	AuthService	Login — cuenta desactivada	Alta

CP-AUTH-05	AdministradorService	Cambio de contraseña exitoso	Alta
CP-AUTH-06	AdministradorService	Cambio — contraseña actual incorrecta	Alta
CP-AUTH-07	AdministradorService	Cambio — confirmación no coincide	Alta
CP-AUTH-08	JwtStrategy	Validación token — admin activo	Alta
CP-AUTH-09	JwtStrategy	Validación token — admin inactivo	Alta
CP-AUTH-10	JwtAuthGuard	Ruta pública sin token	Alta
CP-AUTH-11	JwtAuthGuard	Ruta protegida con API Key	Alta
CP-AUTH-12	JwtAuthGuard	Ruta protegida sin credenciales	Alta
CP-AUTH-13	Zod Schema	Login válido	Media
CP-AUTH-14	Zod Schema	Login — email malformado	Media
CP-AUTH-15	Zod Schema	Cambio contraseña — no coincide	Media
CP-CLI-01	ClienteService	Crear cliente exitoso	Alta
CP-CLI-02	ClienteService	Crear — teléfono duplicado	Alta
CP-CLI-03	ClienteService	Crear — email duplicado	Alta
CP-CLI-04	ClienteService	FindOrCreate — cliente existente	Alta
CP-CLI-05	ClienteService	FindOrCreate — cliente nuevo	Alta
CP-CLI-06	ClienteService	Obtener por ID existente	Media
CP-CLI-07	ClienteService	Desactivar cliente ya inactivo	Media
CP-CLI-08	ClienteService	Actualizar — teléfono en uso	Alta
CP-CLI-09	Zod Schema	Crear cliente válido	Media
CP-CLI-10	Zod Schema	Crear — nombre muy corto	Media
CP-CLI-11	Zod Schema	Crear — teléfono muy corto	Media
CP-CLI-12	Zod Schema	Crear — email inválido	Media
CP-CLI-13	Zod Schema	Filtros — estado válido	Baja
CP-OVW-01	DashboardService	Overview con datos completos	Alta

CP-OVW-02	DashboardService	Variación positiva vs mes anterior	Alta
CP-OVW-03	DashboardService	Variación negativa vs mes anterior	Alta
CP-OVW-04	DashboardService	Variación con mes anterior en cero	Media
CP-OVW-05	DashboardService	Overview sin pedidos del día	Media
CP-OVW-06	DashboardService	Productos stock crítico	Alta
CP-OVW-07	DashboardService	Pedidos recientes ordenados	Media
CP-OVW-08	Zod Schema	Overview válido	Media
CP-OVW-09	Zod Schema	Tendencia inválida	Baja

Total: 37 casos de prueba

- Alta prioridad: 24
- Media prioridad: 11
- Baja prioridad: 2